

IoT Project Report

Pablo Chen

18 December 2021

BLE PoS

BLE PoS is Point of Sale made for the subject IoT of the Universitat Autònoma de Barcelona. The system uses a single board to obtain the data (called the device), an Android application (the computing edge) to process the data and a web API (the cloud server) to store and retrieve the data.

Architecture overview

At a high level, the system has three layers:

- The first layer, the **device** has a single board that will act as a source of the images for the next layer.
- The second layer, the **edge** is a Android application that will connect to the device using BLE and act as a controller for the board.
- The third layer is the **cloud server**. It implements a REST API using Fast API and processes the images using neural networks.

Device

For the device, we ended up using the single board nRF52840 from Nordic Semiconductors which we used to implement a serial port over BLE based on the [example](#) provided by the [BLEPeripheral](#) library to send images to the mobile application.

The nRF52840 exposes one service with uuid `6E400001-B5A3-F393-E0A9-E50E24DCCA9E` and two characteristic `TX` and `RX`, one is used to transmit data the other one is used to receive data.

- RX - Receive Data (Write) uuid: `6E400002-B5A3-F393-E0A9-E50E24DCCA9E`
- TX - Transfer Data (Notify) uuid: `6E400003-B5A3-F393-E0A9-E50E24DCCA9E`

Since the nRF52840 cannot send more than 20 Bytes of data in each packet (defined in `BLE_ATTRIBUTE_MAX_VALUE_LENGTH`) we implemented a simple function that will transmit all packet until all are succesful.

```
while (file_size != file_pos) {
    // handle file packet size and other stuff
    if (packet_size > 0) {
        return_code = this->_txCharacteristic.setValue(&buffer[file_pos], packet_size);
        BLEPeripheral::poll();
        if (return_code == SUCCESS) {
            file_pos += packet_size;
        }
    } else {
        file_size = 0;
        Serial.println("Done transfer");
        break;
    }
}
```

Also considering that the application has to know what is the size of the memory it has to allocate for the buffer, we first send a header before sending the actual file. The header contains the size of the file and CRC32 code for soft-error checking and a OP code used to tell the type of operation to the edge.

OP CODE 1B	Size in Bytes 4B	CRC32 4B
-------------------	-------------------------	-----------------

And finally since we don't have a camera for the nRF, we hardcoded the images in char arrays like this:

```
#define PICTURE_LEN 16709
const uint8_t PICTURE[] = {
    0xFF, 0xD8, 0xFF, 0xE0, 0x00,
    ...
}
```

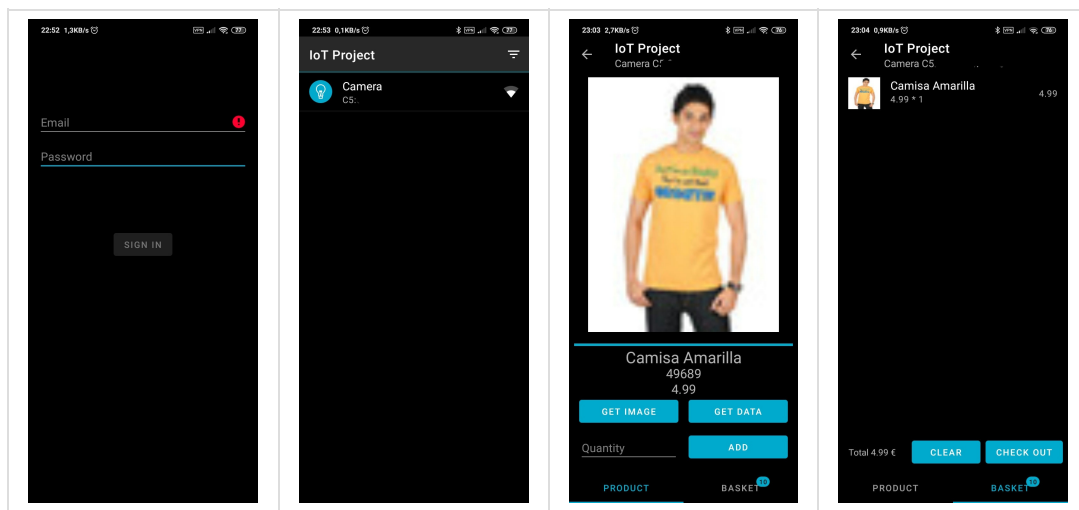
Edge-device

We took as a template the [nRF Blinky](#), and added the logic to receive images inspired by [Android-Image-Transfer-Demo](#).

Interface

The interface consists of various activities and fragments:

- Login activity: Launcher activity, it asks the user for a username and a password. If the password is correct, it launches the next activity.
- Scanning activity: It displays all the discovered Bluetooth devices, there's an option to hide all devices which the application doesn't support.
- Fragment activity: Main activity, it uses ViewPager2 to display two fragments.
 - Product fragment: The fragment that is responsible for displaying information related to the product. It has an ImageView, buttons, textviews and a progress bar to display the state of the transfer.
 - Basket fragment: It implements a RecyclerView to hold all products the user wants to buy.



Bluetooth connection and data management

To connect to the device we extended the class `ObservableBleManager`, which is responsible for handling the connection between the android OS and the nRF board. The class also requires a `BleManagerGattCallback` to implement the verification of the services.

- `isRequiredServiceSupported` which validates if the required services and characteristics are present in the device we are attempting to connect. Also gets a handle to said characteristics.
- `initialize` sets the characteristics callbacks for writing data and reading data. Here we pass the class `DataCallback` that will handle logic for getting the images from the board.

- `onServicesInvalidated` invalidates the handles set by `isRequiredServiceSupported`.

The class `DataCallback` implements the functions which are called when sending and receiving data. It will also hold the buffers needed for the images.

The `onDataSent` is straightforward to implement since we actually don't send any complex data. However the `onDataReceived` has to implement the logic for parsing the header and handling the packets.

The process to request a image from the board is as follows:

- The applications sends a OP code indicating the operation.
- The board sends the first packet consisting of the header.
- The applications checks for the OP code and parses the header and creates a buffer for holding the incoming packets. Also parses the hash.
- From this point, each packet received will be saved in the buffer until it reaches the size.
- Finally, after the transfer is completed we need to parse the bytes as image.

```
val received: ByteArray = data.value!!
if (mBytesTransferred == mBytesTotal && received[0] == 0xFF.toByte()) {
    // parse header and initialize buffer and other variables
    mBytesTransferred = 0
    mDataBuffer = ByteArray(mBytesTotal)
    // over variables
} else {
    val txValue: ByteArray = data.value!!
    mBytesTransferred += txValue.size
    System.arraycopy(txValue, 0, mDataBuffer, mBytesTransferred, txValue.size)
    // some other stuff like updating progress bar
    if (mBytesTransferred >= mBytesTotal) {
        // some other validation and convert buffer to bitmap
        image.value = BitmapFactory.decodeByteArray(mDataBuffer, 0, mDataBuffer.size)
    }
}
```

Communication with the server

The cloud server exposes various REST API endpoints which the app can use to communicate with the server using the library Volley.

Login

To authenticating the user, the app uses a token based approach. The app request using POST the username and password to `/token` and the server responds with the token.

```
val response = Response.Listener<String> {
    Log.i(TAG, it.toString())
    val r = JSONObject(it)
    user = LoggedInUser(r.getString("access_token"), "")
    responseOK.processFinish(it)
}
val error = Response.ErrorListener {
    Log.i(TAG, it.toString())
    responseERROR.processFinish(it.toString())
}
val req = object : StringRequest(Method.POST, url, response, error) {
    @Throws(AuthFailureError::class)
    override fun getParams(): Map<String, String> {
        val params: MutableMap<String, String> = HashMap()
        params["username"] = username
        params["password"] = password
        return params
    }
}
```

Product data

To send the image the app uses `VolleyMultipartRequest` to send *multipart/form-data* content to the endpoint `/items/upload`.

```
val request = object: VolleyMultipartRequest(Method.POST, url, onResponse, onError, {
    override fun getParams(): Map<String, String> {
        val params: MutableMap<String, String> = HashMap()
        params["token"] = getToken()
        return params
    }
    override fun getByteData(): Map<String, DataPart> {
        val params: MutableMap<String, DataPart> = HashMap()
        val image = Utils.getFileDataFromDrawable(binding.ivImageCanvas.drawable)
        params["file"] = DataPart("item.jpg", image, "image/jpeg")
        return params
    }
})
```

A succesful request will result in a JSON containing data about the product.

```
{
  "name": "string",
  "price": 0,
  "image_id": 0,
  "id": 0
}
```

Basket

To upload the customers basket to the server, first we need to transform our *MutableList* in to a *JSON string*, and then POST to `/users/me/orders/`.

```
fun getAsJSON(): JSONObject {
    val orderDetails = JSONArray()
    for (i in products) {
        val item = JSONObject()
        item.put("item_id", i.id)
        item.put("quantity", i.quantity)
        orderDetails.put(item)
    }
    val order = JSONObject()
    order.put("order_details", orderDetails)
    Log.d(TAG, order.toString())
    return order
}

val req: JsonObjectRequest =
    object : JsonObjectRequest(url, productListAdapter.getAsJSON(), response, error) {
        override fun getHeaders(): MutableMap<String, String> {
            super.getHeaders()
            val params: MutableMap<String, String> = HashMap()
            val user = LoginRepository.user
            val token = if (user != null) user!!.access_token else ""
            params["Authorization"] = "Bearer $token"
            return params
        }
    }
}
```

A succesful request will result with the server responding with the uploaded JSON and the `order_id`

and some other data.

```
{
  "order_details": [
    {
      "item_id": 0,
      "quantity": 1,
      "items": {
        "name": "string",
        "price": 0,
        "image_id": 0
      }
    }
  ],
  "id": 0,
  "user_id": 0,
  "datetime": "2021-12-17T23:07:08.341Z",
  "total": 0
}
```

Cloud Server

The server is made using Python3 and the module FastAPI for the endpoints. It uses sqlite as database and TensorFlow for the neural networks.

Database

The database composed of three tables and one association table and uses SQLAlchemy as the database library.

- Users table: It hold the data related to the user. It has a relationship on to many with orders.
- Orders table: Contains information about every order that is made. It has a many to many relationship with items.
 - Orders details: The association table used to relate orders and items. It also hold the quantity of each item in order.
- Items table: Contains information about the items.

```
class Orders(Base):
    __tablename__ = "orders"
    id = Column(Integer, primary_key=True, index=True)
    user_id = Column(Integer, ForeignKey("users.id"), index=True)
    datetime = Column(DateTime(timezone=True), server_default=func.now())
    order_details = relationship("OrderDetails", cascade="all, delete", lazy='selectin',
                                primaryjoin="Orders.id == OrderDetails.order_id")

class OrderDetails(Base):
    __tablename__ = "order_details"
    order_id = Column(Integer, ForeignKey("orders.id"), primary_key=True, index=True)
    item_id = Column(Integer, ForeignKey("items.id"), primary_key=True, index=True)
    quantity = Column(Integer, nullable=False)
    items = relationship("Items", lazy='selectin', primaryjoin="OrderDetails.item_id == Items.id")
```

Data Acces Layer

For sort DAL is the class used to handle all operation on the DB. It implements operations such as `create_user` or `create_order` used to register a user or create an order, `get_all_orders_from_user` used to display all orders from the user.

```
async def create_order(self, user_id: int, order: schemas.OrderBase):
    new_order = models.Orders(user_id=user_id)
    new_order.order_details = [models.OrderDetails(**i.dict()) for i in order.order_details]
```

```

self.db.add(new_order)
await self.db.flush()
await self.db.refresh(new_order)
return new_order

```

To use the class we use a generator passed as dependency to FastAPI.

```

async def get_dal():
    async with async_session() as session:
        async with session.begin():
            yield DAL(session)

@user_router.post("/users/", response_model=schemas.User)
async def create_user(user: schemas.UserCreate, db: DAL = Depends(get_dal)):
    # ...

```

Object-Relational Mapping

To convert the *database tables* (select * from users) to *data objects* (a list of python objects) we used Pydantic. All the models are defined in the `schemas.py` file.

```

class OrderDetailsBase(BaseModel):
    item_id: int
    quantity: int = 1
    items: ItemBase = None

    class Config:
        orm_mode = True

class OrderBase(BaseModel):
    order_details: List[OrderDetailsBase]

    class Config:
        orm_mode = True

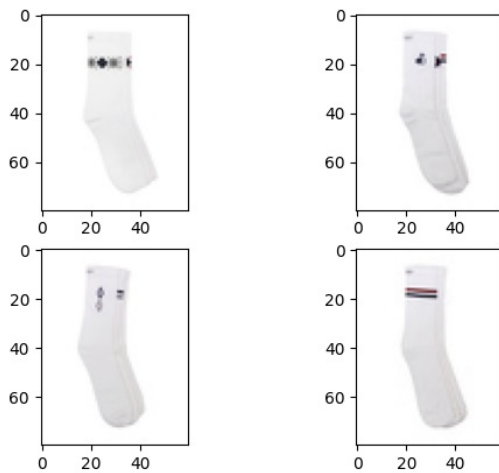
class Order(OrderBase):
    id: int
    user_id: int
    datetime: Datetime
    total: float = 0

    class Config:
        orm_mode = True

```

Image Based Search

On the user uploads a image, the server will search the best matching product and respond with the price and name. The class `SearchImage` handles all the operations related to the neural networks, from the initialization to the searching of images. The class, first extracts features of training image set using a convolutional network (VGG16). To search the image uploaded, we first extract the features of the image and use KNN to obtain the most similar product. To speed up the KNN classification, we create a index of the features using [AnnoyIndex](#). The extraction of the features and the creation of the index are in fact really time consuming, to solve this issue, we instead, initialize once the class and save the features and the items on disk, after the first initialization, each subsequent initialization uses the preprocessed data on disk.



User authentication

The server uses OpenAPI (integrated in FastAPI) specification to handle authentication and authorization, specifically it uses JWT tokens.

Frontend

The frontend shows the latest orders in a table. We use Pydantic to convert the objects to JSON objects and the module json2html to convert JSON to HTML.

order_details				id	user_id	datetime	total
item_id	quantity	items		3420748	501	2021-12-13T11:15:34	109.78
49689	22	name	Camisa Amarilla				
		price	4.99				
		image_id	85				
item_id	quantity	items		3420747	501	2021-12-13T11:15:21	109.78
49689	22	name	Camisa Amarilla				
		price	4.99				
		image_id	85				
item_id	quantity	items		3420746	501	2021-12-13T11:12:11	54.89
49689	11	name	Camisa Amarilla				
		price	4.99				
		image_id	85				

```
@data_router.get("/")
async def get_latest_orders(skip=0, limit=10, db: DAL = Depends(get_dal)):
    orders = await db.get_all_orders(skip=skip, limit=limit, desc=True)
    orders: List[schemas.Order] = parse_obj_as(List[schemas.Order], orders)
    for i in orders:
        for j in i.order_details:
            i.total += j.items.price * j.quantity
    json = jsonable_encoder(orders)
    table = json2html.convert(json=json)
    html = f"""
    <!DOCTYPE html>
    <html><body>{table}</body></html>
    """
    return HTMLResponse(html)
```